Dissertation on

Predictive and Uncertainty-Aware Scheduling of Apache Iceberg Storage Maintenance Under Concurrent Analytical Workloads

Submitted in partial fulfilment of the requirements for the award of the degree

Bachelor of Technology

in

Computer Science and Engineering

UE24CS320A – Capstone Project Phase 1

PW_25_PKA_02

Submitted by

Shashank Keshava Murthy PES1UG24CS434

Shashank D PES1UG24CS433

Prachi Jalan PES1UG24CS326

Samarth Rajesh Rao PES1UG24AM244

Under the guidance of

Dr. Prafullata Kiran Auradkar

Department of Computer Science and Engineering

PES UNIVERSITY

August – December 2026

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

FACULTY OF ENGINEERING

PES UNIVERSITY

(Established under Karnataka Act No. 16 of 2013)

100 Feet Ring Road, BSK 3rd Stage, Hosakerehalli, Bengaluru – 560085

PES UNIVERSITY

FACULTY OF ENGINEERING

CERTIFICATE

This is to certify that the dissertation entitled “Predictive and Uncertainty-Aware Scheduling of Apache Iceberg Storage Maintenance Under Concurrent Analytical Workloads” is the work submitted by the following students in partial fulfilment of the requirements of Fifth-semester Capstone Project Phase 1 (UE24CS320A), PES University, Bengaluru, during August – December 2026.

Shashank Keshava Murthy PES1UG24CS434

Shashank D PES1UG24CS433

Prachi Jalan PES1UG24CS326

Samarth Rajesh Rao PES1UG24AM244

The report is submitted for guide and examination review. Approval of the dissertation, incorporation of required corrections and the final certificate are subject to the signatures below. Signatures and approval dates are to be completed by the authorised signatories.

Guide signature: _____ Date: _____

Dr. Prafullata Kiran Auradkar

Designation: _____

Chairperson signature: _____

Dr. Shylaja S S

Dean of Faculty and Registrar signature: _____

Dr. K S Sridhar

External viva examiner(s) and signature(s):

_____

_____

DECLARATION

We declare that this report, “Predictive and Uncertainty-Aware Scheduling of Apache Iceberg Storage Maintenance Under Concurrent Analytical Workloads”, describes our Capstone Project Phase 1 work carried out under the guidance of Dr. Prafullata Kiran Auradkar. It is submitted in partial fulfilment of the Bachelor of Technology course requirements at PES University during August – December 2026. The report distinguishes implemented modules, exploratory results, failed experiments and future work. External sources and the project evidence used in the report are identified.

The work presented here has not been submitted by us to another institution for the award of a degree. This declaration is to be confirmed by the student signatures below.

Shashank Keshava Murthy PES1UG24CS434
Signature: _____

Shashank D PES1UG24CS433
Signature: _____

Prachi Jalan PES1UG24CS326
Signature: _____

Samarth Rajesh Rao PES1UG24AM244
Signature: _____

ACKNOWLEDGEMENT

We thank Dr. Prafullata Kiran Auradkar for guiding this project and encouraging us to question whether the proposed models improve a useful system metric. That feedback helped us focus on reliable evidence and a clear explanation of the work completed.

We thank the Capstone Project Coordinator, Prof. Jayashree S, the Department of Computer Science and Engineering, and PES University for the support provided during Phase 1. We are grateful to Dr. Shylaja S S and Dr. K S Sridhar for the academic environment in which this work was carried out.

We also acknowledge the maintainers of Apache Spark, Apache Iceberg, TPC-H and Snowset. Their software and documented data sources made it possible to build an inspectable experimental setup. We thank our families and teammates for their support throughout the work.

ABSTRACT

Data-file compaction combines small files so that an analytical system has fewer files to manage. The operation is useful, but it also consumes resources while queries are running. Our project investigates whether workload forecasts and queue information can help choose a better time to compact an Apache Iceberg table. We use Spark SQL and Iceberg as the execution platform, TPC-H data and selected SQL templates as the controlled workload, and recorded Snowset timestamps to determine when requests arrive. This is a trace-driven benchmark mapping rather than a replay of Snowflake SQL.

Phase 1 has established a remote testbed, data and layout checks, query and queue measurement, exploratory cost estimators, and an evidence archive with explicit acceptance gates. In a historical SF1 reserved-family evaluation, the query/layout median had a mean absolute error of 20.34 ms, compared with 20.93 ms for a shallow tree and 21.54 ms for log Ridge. The learned query models did not improve on the strongest simple baseline in that evaluation. At SF10, eight tables were ingested and a separate capability run reduced a 64-file layout to four files. A later two-arm validation campaign recorded 463 completed calls in its control arm but failed the attribution audit; its maintenance arm was not attempted. A subsequent diagnostic linked all 308 rejected job references to their exact marked SQL queries. This finding identifies an audit defect and does not make the failed campaign acceptable for training.

The present contribution is a working experimental foundation and an honest account of the limits of the evidence. The next stage is to repair prospective attribution, complete a fresh audited control/maintenance comparison, and evaluate timing policies on independent workload families. Calibrated uncertainty and a scheduling advantage remain goals rather than demonstrated results.

TABLE OF CONTENTS

1 Introduction..... 1

2 Problem Definition..... 3

3 Literature Survey..... 5

4 Research And Technology Gaps And Challenges..... 7

5 Requirement Specification..... 9

6 Proposed System Architecture..... 11

7 Implementation..... 13

8 Results And Implementation Evidence..... 16

9 Conclusion Of Capstone Project Phase 1 And Future Work..... 22

References / Bibliography..... 24

Appendix A Definitions, acronyms and evidence sources..... 25

LIST OF TABLES

Table 2.1 Project objectives and verification....3

Table 3.1 Related work and the question retained for this project....6

Table 5.1 Functional requirements and present status....9

Table 5.2 Recorded execution and authoring environment....10

Table 6.1 Candidate model roles....12

Table 7.1 TPC-H schema and verified SF10 row counts....13

Table 8.1 Saved SF1 reserved-family latency errors....16

Table 8.2 Implemented, observed and pending work....20

Table A.1 Definitions....25

LIST OF FIGURES

Figure 6.1 Workload-aware maintenance architecture.....11

Figure 8.1 Historical SF1 baseline comparison.....17

Figure 8.2 Remote processes at capture time.....18

Figure 8.3 Spark completed-job history.....19

Figure 8.4 Dashboard summary of the failed control.....19

Figure 8.5 Preserved campaign audit status.....20

CHAPTER 1
INTRODUCTION

1.1 Background and motivation

Analytical lakehouse systems store large tables as collections of columnar files. When the same table contains many small files, the engine may spend additional effort finding, opening and planning work over those files. Compaction rewrites the files into a more useful layout. Apache Iceberg exposes this operation through its Spark procedures [1].

The choice of time matters because a maintenance operation and a user query can ask for the same processor, memory and storage resources. A quiet period may be a better opportunity than a burst of queries. However, observing that compaction uses resources is not enough to show that a forecasting model is needed. The model and its controller must improve a measured outcome after their own overhead is included.

1.2 What our project is trying to answer

Our question is whether past workload observations and the current query queue can help choose a compaction time that improves user-visible query response, while accounting for the maintenance work that still has to be completed. A query response includes both its waiting time and its actual execution time. A model that predicts execution time accurately may still make a poor scheduling decision if it ignores the queue.

We keep the scope specific: Spark SQL queries over Iceberg tables backed by Parquet files. We study the timing of data-file compaction. We do not change Spark planner internals, claim a new storage format, or assume that a result transfers automatically to another engine or resource configuration.

1.3 Work completed in Phase 1

We have built and exercised the remote execution stack, ingested a verified SF10 dataset, implemented trace-driven request replay, recorded query and queue events, and trained exploratory SF1 estimators. We also built a read-only dashboard and captured real remote process and Spark UI views for the ESA review. The images are labelled by capture time and distinguish live service observations from historical experiment results.

The main result so far is a clearer measurement and evaluation design. The earlier SF1 comparison showed that a simple query/layout median was difficult to beat. The SF10 collection work exposed problems in telemetry continuity and Spark job attribution. We have preserved those failures and separated successful engineering checks from accepted model evidence.

1.4 How to read this report

Chapters 2 to 5 define the problem, related work, gaps and requirements. Chapter 6 explains the architecture. Chapter 7 describes the implementation and datasets, and Chapter 8 presents the saved results and screenshots. Chapter 9 records what Phase 1 has established and the next work needed. The appendix provides terminology, feature groups and source locations for the evidence.

CHAPTER 2
PROBLEM DEFINITION

2.1 Problem statement

Given an Iceberg table, a stream of analytical query arrivals, current FIFO backlog and observations available before a decision, choose whether to start compaction now or defer it. The decision should improve the response of the complete offered workload without hiding unfinished queries or an unpaid maintenance obligation. Any learned policy must be compared with simpler policies under equivalent initial conditions and a consistent final-state rule.

2.2 Objectives

Table 2.1 Project objectives and verification

Objective	How it is checked
Build a reproducible testbed	Source manifests, verified table contents, registered protocols and fresh experiment identities
Measure workload response	Record offered arrivals, dispatch, completion, waiting and remaining queue IDs
Estimate costs and demand	Compare A/B and shared latency candidates with medians; forecast future arrival counts using past observations
Choose a maintenance time	Evaluate a proposed controller against declared timing baselines on fresh independent blocks
Make results inspectable	Preserve raw events, result receipts, hashes, failed attempts and clearly labelled figures

2.3 Meaning of a fair comparison

The two conditions must start from matched data and layout, receive the same offered requests and use the same accounting horizon. We must also state what happens to work left at the end. Finishing

fewer queries or avoiding compaction can make an arm look inexpensive, but it does not establish an efficiency improvement when the other arm performs more work.

We therefore report completed, queued, failed and unattempted work alongside latency. For future policy experiments, the maintenance obligation and the drain or residual-debt rule must be fixed before outcomes are seen. We do not combine unrelated measurements into a made-up scalar score or assume an SLA that the project has not defined.

2.4 Expected outcomes and boundaries

A positive outcome would be a repeatable response-time improvement from a policy using past-only forecasts, after accounting for controller and maintenance costs. A negative outcome is also meaningful: a simple baseline may be sufficient in the tested regime, or the timing effect may be smaller than ordinary run-to-run variation. Phase 1 has not established a learned scheduling advantage, calibrated prediction intervals or SF100 readiness.

CHAPTER 3
LITERATURE SURVEY

3.1 Table compaction and maintenance

AutoComp studies automated data compaction for log-structured tables in data lakes [2]. It is directly relevant to the practical need to combine files and account for compaction cost. Our project must compare its timing question with existing maintenance mechanisms rather than present file-count reduction itself as a new contribution.

SILK studies interference between user operations and internal maintenance in an LSM key-value store [3]. Its resource scheduling setting differs from an external controller for Spark and Iceberg. It nevertheless provides a useful warning: throughput and user-facing latency are different objectives, and internal work can affect the latter.

3.2 Learned cost prediction and compaction utility

Strausz, Pardon and Giurgiu use query-plan representations in a learned cost model for cross-engine SQL routing [4]. That is a different decision from compaction timing. The work supports examining query structure as an input, but it is not evidence that separate A/B models or a maintenance scheduler will work in our testbed.

Smart Compaction investigates predicting compaction utility from lakehouse metadata [5]. This overlaps with our layout and maintenance-cost concerns. We should not claim that using machine learning with file metadata is novel. Our intended evaluation instead joins queue response, future offered demand and measured maintenance consequences; its value remains to be established experimentally.

3.3 Uncertainty and data sources

Conformal prediction provides a framework for constructing prediction sets under stated assumptions [6]. It does not turn dependent measurements from a single physical rewrite into independent calibration examples. We need separate workload blocks and an appropriate coverage evaluation before calling our model uncertainty-aware in an operational sense.

TPC-H provides a documented decision-support schema and query definitions [7]. Snowset supplies recorded query statistics and timing information [8]. We use these sources for different

purposes: TPC-H defines controlled data and SQL, while Snowset provides arrival spacing. Combining them does not reproduce the original production application.

Table 3.1 Related work and the question retained for this project

Work	Relevant idea	Difference or open issue
AutoComp [2]	Automated lake-table compaction	Timing under our concurrent queue workload still needs direct evaluation
SILK [3]	Control maintenance interference	LSM resource scheduler; different execution and storage setting
Cross-engine learned cost model [4]	Use query plans for prediction	Routes SQL across engines rather than choosing compaction time
Smart Compaction [5]	Predict maintenance utility from metadata	Metadata-based ML alone is not our novelty claim
Conformal prediction [6]	Prediction-set coverage	Independent, relevant calibration blocks are still missing

3.4 Implication for our contribution

The defensible research direction is to evaluate forecast- and queue-aware timing against strong simple alternatives, with complete response and maintenance accounting. We have not completed the literature comparison or physical evaluation needed to claim publication-level novelty. The earlier Review 2 references were treated as leads and checked before reuse; unsupported blanket statements that prior work never considers timing have been removed.

CHAPTER 4
RESEARCH AND TECHNOLOGY GAPS AND CHALLENGES

4.1 Prediction accuracy does not prove scheduling value

Our historical SF1 results already show why this distinction matters. Query identity and file layout can explain enough variation for a median baseline to match or outperform a learned model. A useful next experiment must show that permitted timing choices lead to different outcomes, not simply that another regressor can be trained.

4.2 Independence and condition coverage

Hundreds of query calls from the same physical episode share source history, host conditions and often one rewrite. They cannot be treated as hundreds of independent experiments. In the October 22 exploratory training set, 16 complete-history queries had a pending maintenance request and 319 did not. The conditions also had different layouts. This imbalance and confounding prevent a causal interpretation of their difference.

We split whole episodes and source-overlap families rather than randomly distributing rows. The historical reserved test family has already been evaluated and cannot be reused for tuning. New comparisons need fresh registered families with enough support in the relevant query, layout and maintenance conditions.

4.3 Reliable observation is part of the experiment

The SF10 work uncovered two separate observation issues. One campaign failed telemetry continuity. A later campaign captured a terminal task archive, but its foreign-work gate rejected generated broadcast jobs whose descriptions did not contain our SQL marker. The subsequent diagnostic linked those jobs directly to marked SQL execution records. That diagnosis gives us a specific repair target; it does not supply a missing prospective task-completeness proof.

4.4 Overhead and resource attribution

Telemetry, plan extraction, inference, action comparison and training all cost something. We must measure them separately and include the relevant online work in any claimed benefit. Host CPU percentage is not the same as CPU work attributable to the controller. Spark task CPU must not be

added to the enclosing process CPU as if they were disjoint measurements. No CPU or server-load improvement is claimed in this report.

4.5 Scope of generalization

The present evidence is tied to specific hardware, selected SQL templates, trace windows and prepared layouts. SF10 is a larger test than SF1, but it is not SF100 and is not a certified TPC-H performance submission. A broader claim would require more source windows, query structures, layouts, scales and resource settings.

CHAPTER 5
REQUIREMENT SPECIFICATION

5.1 Functional requirements

Table 5.1 Functional requirements and present status

Requirement	Present implementation / evidence
Replay registered arrivals	FIFO ledger and trace-driven collector; offered requests retained
Execute benchmark SQL and compaction	Spark/Iceberg stack and SF10 capability evidence
Capture pre-query observations	Host-history, table-state and query-plan paths implemented; eligibility audited
Train and compare estimators	Historical SF1 candidates and comparators saved; SF10 fit blocked
Inspect jobs and task attempts	Periodic and terminal archival implemented; prospective SQL ownership repair pending
Display review evidence	Read-only dashboard, raw snapshots and automatic screenshot manifest
Preserve failures and notify	Unique logs/receipts and ntfy failure wrapper; delivery failures remain visible

5.2 Non-functional requirements

Results must be reproducible from named inputs, code revisions and raw artifacts. Inputs unavailable at the decision time must not enter the predictor. Missing histories, task counters or source records remain missing; the pipeline must not silently replace them with zeros or baseline predictions. An unsupported learned prediction returns no value and an explicit reason.

Physical jobs run detached on worker3 and preserve their exit status. Disconnecting the laptop must not cancel remote science. The local screenshot transport has a different reliability boundary: it needs the laptop and VPN, and its failures are recorded. Existing services, failed attempts and data are preserved rather than restarted or deleted to obtain a favourable outcome.

5.3 Hardware and software requirements

Table 5.2 Recorded execution and authoring environment

Component	Recorded configuration or use
Execution host	worker3, accessed over campus SSH/VPN
Spark	Version 3.3.4; recorded backend uses local[16] and 16 GB driver heap
Iceberg	Version 1.4.3 Spark 3.3 / Scala 2.12 runtime
Storage	Hadoop catalog and Parquet files; snapshots and file metadata retained
Learning tools	Python, scikit-learn, NumPy and joblib; exact fit environment in saved model review
Observation	Host counters, Spark REST/UI, durable JSON event ledger and SHA256 receipts
Presentation transport	Read-only server, SSH tunnel and local browser screenshots

The local[16] setting describes configured Spark parallelism, not a verified claim that the experiment ran on a 16-node cluster. The 16 GB setting is the Java driver heap allocation; it is not a statement of total installed server memory.

CHAPTER 6
PROPOSED SYSTEM ARCHITECTURE

6.1 System overview

The architecture separates workload input, physical execution, evidence validation and decision logic. This separation lets us improve a model without changing what counts as a measured query, and it prevents a failed collection run from being treated as successful training data.

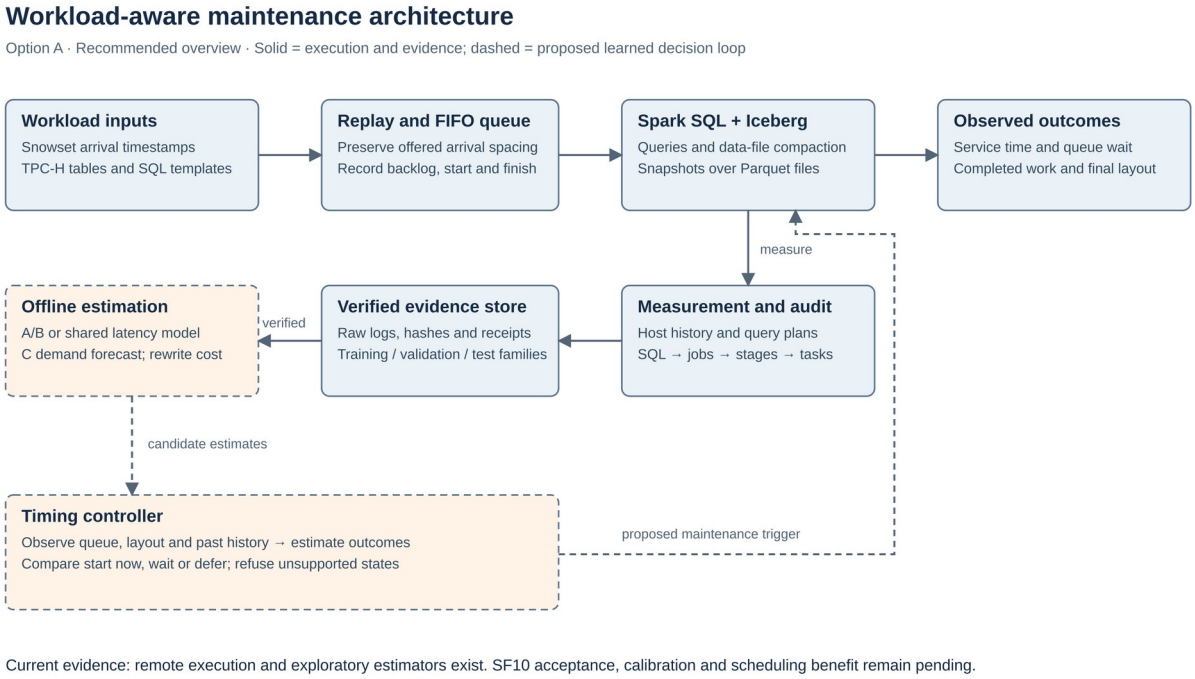


Figure 6.1 Workload-aware maintenance architecture

Solid paths show the execution and evidence foundation. Dashed paths show the exploratory or proposed learned decision loop; they are not a certificate of scheduler readiness.

6.2 Workload and execution modules

The replay module reads the registered arrival sequence and assigns benchmark SQL templates. The FIFO ledger records when a request was offered, queued, dispatched and completed. Spark SQL runs each actual query. Iceberg tracks snapshots and layout while Parquet stores the table data. A separate maintenance request invokes data-file rewriting when an experiment protocol calls for it.

6.3 Observation and evidence modules

The observation path records host counters, table state, plan information, client timings and engine execution evidence. The subprocess sampler acquires counters independently from the collector. Periodic archival and a mandatory terminal archive preserve engine evidence before final inventory and full-result validation. Audits check the raw records before publication or fitting.

The ownership defect shows why the archive must link a marked SQL execution to its declared jobs, then to the corresponding stages and task attempts. A generated job may lack the original SQL text in its description. Prospective repair must use direct engine links and still reject unknown overlapping work; a blanket exception for broadcast jobs would weaken the gate.

6.4 Model responsibilities

Table 6.1 Candidate model roles

Component	Prediction target	Boundary
A	Query service time in milliseconds when the client maintenance request is pending	Pending is not proof of active engine interference
B	Query service time without a pending request, conditional on actual layout	No invented fragmentation or counterfactual labels
Shared comparator	Query service time using maintenance state as an input	Same source splits and allowed predictor information
C	Future offered query-arrival count over a registered horizon	Counts are not CPU load or original Snowflake SQL mix
Rewrite estimator	Rewrite duration and resulting physical transition	Separate measured target; not obtained by subtracting A and B

6.5 Future timing controller

Proposed only: combine queue, layout and estimates to compare actions. A/B artifacts do not establish calibration, causality or scheduling benefit.

CHAPTER 7
IMPLEMENTATION

7.1 Implementation overview

The implementation is organised under three_model_maintenance. Runtime code handles replay, state publication, queue accounting and observation. Preparation code binds actual features and query plans. Models and training contain separate, shared and empirical candidates. Audits reconstruct saved evidence, and protocols record inputs, hashes, parameters and output identities. The read-only observability module displays saved evidence and current service state.

7.2 Dataset description

TPC-H models a wholesale business with customers, orders, order lines, parts, suppliers and geography [7]. Our SF10 preparation receipt records the following actual table row counts. The 61-column schema is used as controlled benchmark data. The SQL workload is six TPC-H-derived templates, not the complete official 22-query suite.

Table 7.1 TPC-H schema and verified SF10 row counts

Table	Columns	Recorded rows	Meaning
region	3	5	Geographical regions
nation	4	25	Countries and region keys
supplier	7	100,000	Supplier records
customer	8	1,500,000	Customer records
part	9	2,000,000	Part descriptions
partsupp	5	8,000,000	Part–supplier associations
orders	9	15,000,000	Order headers
lineitem	16	59,986,052	Individual order lines

Snowset contains recorded query statistics from Snowflake [8]. We use selected arrival timestamps to set the spacing between offered benchmark requests. We do not possess or replay the corresponding production SQL and schemas. Our mapping deliberately imposes a controlled SQL mix, so any conclusion applies to that mapped workload.

The recovered SF10 validation used 3,131 actual offered arrivals over its registered 1,200-second workload horizon. It retained all offered work rather than thinning requests when the server could not keep up. The ledger therefore distinguishes the 463 completed calls from the 2,668 requests still queued. Queued requests have no measured service-time label.

7.3 Proposed methodology and current execution

Each physical experiment begins from named inputs and a registered protocol. The collector checks capacity and existing activity, prepares a fresh isolated experiment identity and verifies source/layout state. It acquires actual host observations, replays the request stream and records maintenance state as it was available at feature time. After the workload threads finish, terminal archival precedes cooperative observer completion and final correctness checks.

Acceptance is a separate step from completion. Logical data checks, raw engine records, task attempts, telemetry history, queue conservation, snapshot state and checksums must agree. Failed attempts remain in the ledger, dependent stages stop, and incomplete data cannot be upgraded into labels merely because Spark returned results.

7.4 Features and candidate estimators

The basic query contract contains 14 predictors: query identity; file count, row count and bytes; pending maintenance state; queue depth; and CPU, memory, read-rate and write-rate summaries for both 60-second and 300-second histories. The SF10 plan-aware candidate adds eight operator counts: total operators, scans, joins, aggregates, exchanges, sorts, filters and windows. These make 22 registered candidate inputs, not evidence that an SF10 model has been trained.

The October 22 exploratory fit used the 14-feature contract and 335 complete-history observations. It saved log-target Ridge and shallow-tree bundles, each with A, B and shared estimators. SQL, SQL/layout and SQL/layout/state medians were independent comparators. Their predictions do not silently replace an unsupported learned prediction. In that fit, one development family and confounded layouts prevented a held-out generalization claim.

7.5 Tools, monitoring and automatic capture

Standalone worker jobs use the failure wrapper with a unique label, receipt and log. The wrapper retains the original exit status even if ntfy delivery fails. The dashboard reads actual process output, Spark REST and saved event records. The screenshot tool opens real pages in a browser, stores images and source snapshots, and records image hashes in a manifest. Spark monitoring and REST endpoints are documented in [9].

The review session produced two sets of four views on November 11. No experiment was running at the time of those captures: the live Spark job count was zero. The browser recorder later timed out when transport was lost; its alert also could not reach the network. Those delivery failures are retained. They do not establish that the remote engine failed.

CHAPTER 8
RESULTS AND IMPLEMENTATION EVIDENCE

8.1 Historical SF1 prediction comparison

The historical SF1 corpus contained 18 physical episodes, 9,923 query labels and 12 rewrite labels across three source families. Those query rows are not independent experimental repetitions. The October 17 evaluation used one reserved source family, with 1,663 eligible query observations across six episodes. Models were not refitted before that evaluation. Equal-episode mean absolute error first averages the absolute prediction error within each episode, then gives each episode equal weight. A lower number means closer service-time predictions; it does not measure the benefit of a timing policy.

Table 8.1 Saved SF1 reserved-family latency errors

Method	Equal-episode MAE (ms)
Query/layout median	20.3430
Shallow tree	20.9283
Query median	21.2335
Log Ridge	21.5426

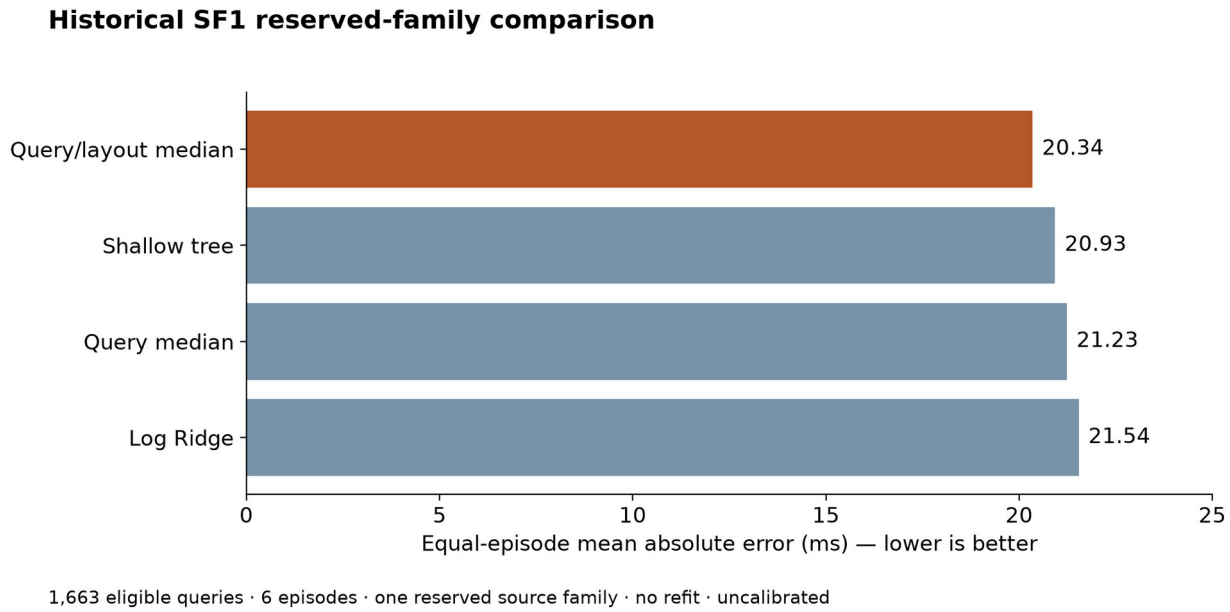


Figure 8.1 Historical SF1 baseline comparison

Replotted from the saved October 17 evaluation. These are historical service-time errors, not new results or scheduler improvements.

The query/layout median was the strongest comparator in this evaluation. This is a useful negative result: a more complex estimator was not justified by these data alone. The test family is consumed and cannot be reused to choose a new model. The four reserved rewrite labels were exploratory and are too limited for a broad rewrite-model conclusion.

8.2 Exploratory A/B artifacts and SF10 capability

The October 22 pipeline completed source verification, specification freeze, fitting and artifact reload checks. It used 16 pending-maintenance and 319 nonpending observations from one development family. This established that the estimator pipeline could produce and reload real candidate models. It did not establish independent accuracy, calibrated uncertainty or causal interference.

A separate SF10 capability run recorded a rewrite from 64 files to four, with 1,714,218,564 bytes reported as rewritten. This is evidence that data-file compaction executed and changed the physical layout. It is separate from the later failed control campaign. File reduction alone is not a query-speedup measurement.

8.3 Recovered SF10 campaign and diagnostic

The November 1 archive03 campaign ended with zero passed episodes, one failed control episode and one unattempted maintenance episode. It captured terminal engine evidence, but its final audit

reported foreign Spark work. The November 11 diagnostic matched all 463 completed calls to unique exact marked SQL execution records and linked all 308 rejected job references directly to those executions. There were no unresolved queries or rejected links in that diagnostic.

The diagnosis supports replacing the description-only attribution rule with prospective SQL-to-job ownership capture. It does not prove complete stage/task coverage and does not retroactively pass archive03. The 308 figure counts ownership links, not 308 independent experiments. No SF10 fitted or calibrated model follows from this result.

8.4 Screenshot evidence for the ESA review

The four views below use the later capture group, approximately 22:26 IST on November 11. Their full originals, earlier captures and SHA256 manifest accompany this report. Exact crops remove empty space for readability and are labelled as crops; they do not change the source content.



Figure 8.2 Remote processes at capture time

Exact crop of B01. Spark and the review server were running on worker3. This is service-state evidence, not a running training or compaction process.

This image is useful when explaining where the work runs. The Python wrapper launches the Spark backend, the Java process is Spark itself, and the review server exposes read-only monitoring. The long command includes Iceberg extensions, the warehouse path and the recorded resource configuration.

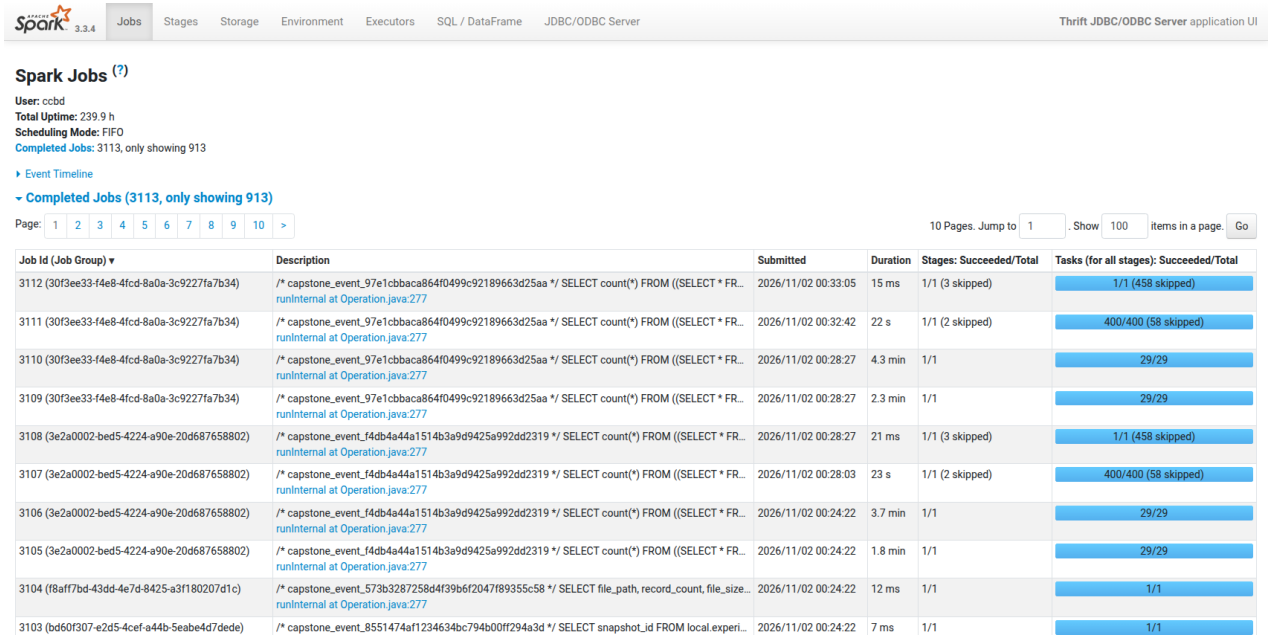


Figure 8.3 Spark completed-job history

Exact crop of B02. The UI shows real historical SQL jobs, durations, stages and task counts. Application-wide completed-job totals are not experiment counts.

The job table shows that actual SQL reached the engine and generated execution tasks. It can support a demonstration of the runtime and its inspection tools. It cannot certify our whole experiment, because the independent audit can fail even when individual Spark jobs succeeded.

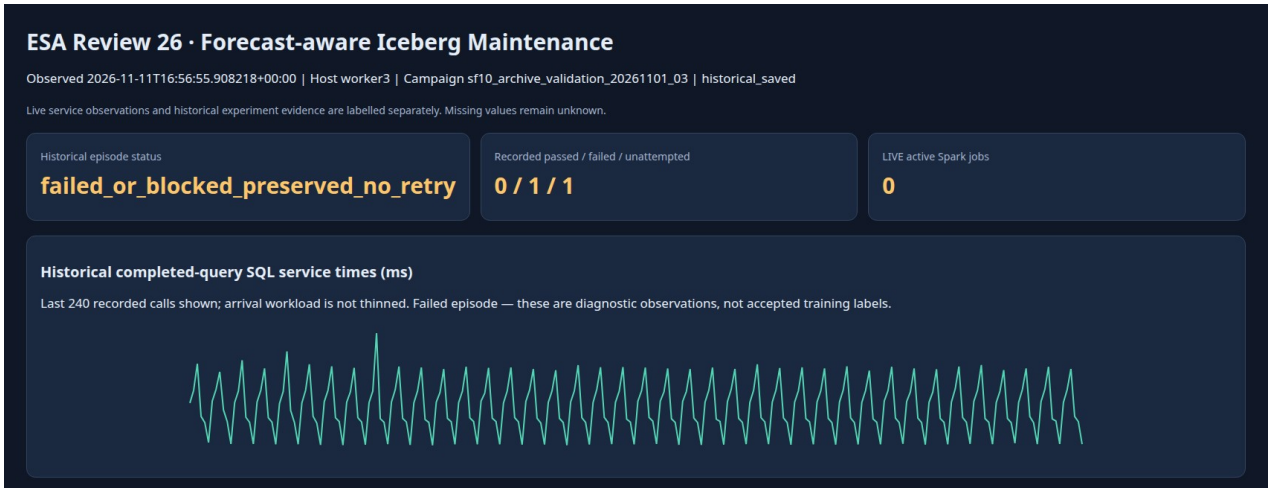


Figure 8.4 Dashboard summary of the failed control

Exact crop of B03. Historical status is 0 passed / 1 failed / 1 unattempted; live active Spark jobs were zero. The line is recorded SQL service time, not a learned prediction.

The completed and queued counts tell the reviewer that we retained offered requests rather than dropping the backlog. The control layout stayed at 64 files, which is expected in a no-maintenance

arm. The separately labelled 64-to-four capability shown in the full dashboard belongs to an earlier run.

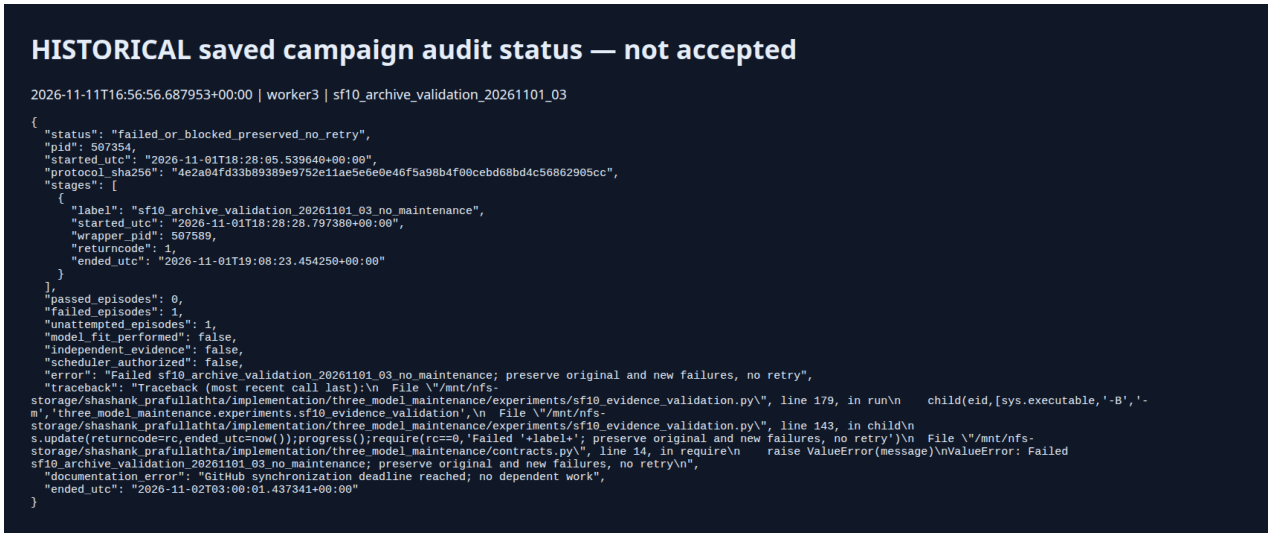


Figure 8.5 Preserved campaign audit status

Exact crop of B04. The ledger records failure and withholds model fitting and scheduler authorization. It is not a pass certificate.

This final screenshot explains how we decide whether measurements are usable. A result is accepted only when the required evidence passes its checks. The later ownership diagnosis is progress in making that check correct, while the original failed outcome remains unchanged.

8.5 What we can demonstrate at this review

Table 8.2 Implemented, observed and pending work

State	Evidence or limitation
Implemented	Remote Spark/Iceberg execution, replay ledger, observation archive, candidate trainers and read-only dashboard
Observed in historical evidence	SF1 baseline comparison; exploratory A/B artifacts; SF10 ingestion and separate 64-to-four capability
Failed and preserved	SF10 archive03 control audit; maintenance arm unattempted
Diagnosed, not repaired in a fresh run	463 exact SQL matches and 308 direct rejected-job ownership links

State	Evidence or limitation
Uncalibrated / not accepted	Candidate estimators and proposed learned scheduling loop
Next gate	Prospective attribution repair, complete raw/task audit and fresh independent control/maintenance comparison

CHAPTER 9
CONCLUSION OF CAPSTONE PROJECT PHASE 1 AND FUTURE WORK

9.1 Phase 1 conclusion

Phase 1 has moved the project from a scheduling idea to an executable, inspectable testbed. We can prepare the benchmark tables, execute queries and compaction remotely, retain a FIFO workload ledger, train exploratory estimators from real measurements, and show the resulting evidence. The screenshot package makes this progress visible without implying that an idle service is an active experiment.

The results also narrowed the research claim. A simple median beat the learned query estimators in the historical SF1 test. SF10 compaction capability was demonstrated, but the later paired validation did not pass. The direct ownership diagnostic explains a specific attribution defect and provides a concrete next engineering task. We have not yet shown that forecasting improves maintenance timing.

9.2 Next implementation and evaluation gates

First, implement and validate prospective SQL execution ownership, including all declared jobs, stages and task attempts, while retaining the foreign-work guard. Second, register fresh matched control and compaction identities, source windows, resource conditions and cutoffs. Run the pair only after admission checks pass; preserve all failures and unattempted counts.

Third, export only accepted measurements and compare simple medians, shared estimators and separate A/B candidates under identical whole-family splits. Query-plan features and scale variation should be evaluated through declared ablations rather than added until a favourable score appears. C forecasting should be tested against past-only persistence and measured for overhead.

Finally, establish that timing choices have a useful repeatable effect before evaluating a learned policy. The comparison must include waiting, service, completed work, remaining maintenance and online overhead. Calibrated uncertainty needs separate relevant blocks. CPU or I/O benefits need attributable accounting, not a conversion from utilisation percentages.

9.3 Response to review feedback

The Review 1 presentation records a request to explain TPC-H structure. This report now describes all eight tables, their column counts, verified SF10 row counts and the separate roles of Snowset and benchmark SQL. For Review 2, the team reports that the panel asked to see actual screenshots and an explanation of what work had started. Chapter 8 and the accompanying presentation address that request directly. No additional panel remark has been invented.

REFERENCES / BIBLIOGRAPHY

[1] Apache Iceberg, “Spark procedures,” version 1.4.3 documentation, data-file rewriting. https://iceberg.apache.org/docs/1.4.3/spark-procedures/ .

[2] A. Gruenheid et al., “AutoComp: Automated Data Compaction for Log-Structured Tables in Data Lakes,” 2025, arXiv:2504.04186. https://arxiv.org/abs/2504.04186 .

[3] O. Balmau et al., “SILK: Preventing Latency Spikes in Log-Structured Merge Key-Value Stores,” USENIX ATC, pp. 753–766, 2019. https://www.usenix.org/conference/atc19/presentation/balmau .

[4] A. Strausz, N. Pardon and I. Giurgiu, “A Learned Cost Model-based Cross-engine Optimizer for SQL Workloads,” 2025, arXiv:2506.02802. https://arxiv.org/abs/2506.02802 .

[5] J. Cutura and S. Prakash, “Smart Compaction: Predicting Compaction Utility from Lakehouse Table Metadata,” 2026, arXiv:2608.08639, preprint. https://arxiv.org/abs/2608.08639 .

[6] G. Shafer and V. Vovk, “A Tutorial on Conformal Prediction,” Journal of Machine Learning Research, vol. 9, pp. 371–421, 2008. https://www.jmlr.org/papers/v9/shafer08a.html .

[7] Transaction Processing Performance Council, “TPC Benchmark H Standard Specification,” revision 3.0.1, 2022. https://www.tpc.org/tpc_documents_current_versions/pdf/tpc-h_v3.0.1.pdf (schema reference; our experiment is not a certified TPC-H result).

[8] Resource Disaggregation group, “Snowflake Dataset (Snowset),” dataset documentation accompanying Building an Elastic Query Engine on Disaggregated Storage. https://github.com/resource-disaggregation/snowset .

[9] Apache Spark, “Monitoring and Instrumentation,” version 3.3.4 documentation. https://dlcdn.apache.org/spark/docs/3.3.4/monitoring.html .

APPENDIX A
DEFINITIONS ACRONYMS AND ABBREVIATIONS

A.1 Terms used in the report

Table A.1 Definitions

Term	Meaning
Compaction	Rewrite several data files into a different physical layout
FIFO	First in, first out request dispatch
Service time	Measured execution-and-fetch interval for a query call
Response time	Time from offered arrival until result; includes queue waiting
MAE	Mean absolute prediction error in the target unit
Source family	A group connected by overlapping source windows or shared experiment history
Pending maintenance	The client has requested maintenance and has not observed completion
Accepted evidence	Measurements that pass the registered audit gates
Candidate model	Exploratory estimator without calibrated or scheduling authorization
SF1 / SF10	Benchmark generation scales; not literal compressed table sizes
SHA256	A file digest used to bind exact evidence bytes

A.2 Evidence locations

Paths below are relative to the implementation repository. They identify saved project evidence, not third-party publications. The review package uses a frozen local snapshot and does not claim current remote process state.

SF1 held-out comparison

results/consistent_reserved_evaluation_20261017/result.json

Saved historical review

docs/final_review_2026-10-17.md

Exploratory A/B fit review

three_model_maintenance/docs/reviews/FIRST_DEVELOPMENT_FIT04_2026-10-22.md

SF10 verified tables

results/three_model_sf10_scratch_recovery_20261025_03/prepare/result.json

Separate SF10 compaction capability

results/three_model_sf10_scratch_recovery_20261025_03/plan_query/maintenance.json

Failed campaign ledger

results/sf10_archive_validation_20261101_03/result.json

Post-hoc SQL ownership diagnostic

results/sf10_sql_ownership_diagnostic_20261111_01/result.json

Screenshot capture ledger

docs/Review/ESA review 26/screenshots/capture_manifest.jsonl

Candidate feature contract

three_model_maintenance/contracts.py

Plan-aware training contract

three_model_maintenance/training/sf10_comparison.py

A.3 Screenshot provenance

Session A was captured at approximately 20:15 IST and session B at 22:26 IST on November 11, 2026. Eight PNGs cover four view types. Full images and SHA256 values are in Labelled evidence/evidence_index.json. Crops in this report are derived only for display. Neither capture session shows an active physical experiment.

A.4 Report status and remaining administrative fields

Prepared for the ESA review from saved evidence available on November 13, 2026. The title, project ID and guide name follow the submitted Review 2 presentation. The team names and SRNs follow the details supplied by the team. Guide designation, dates and signatures require human completion. Contact details and CGPAs have intentionally not been included.